

Kode ini mengimplementasikan model klasifikasi gambar menggunakan arsitektur MobileViT (XXS variant) dengan Keras (menggunakan backend TensorFlow). Berikut adalah analisisnya:

- 1. Instalasi dan Import:** Kode dimulai dengan menginstal library `mediapipe` (meskipun tidak secara langsung digunakan dalam blok kode yang membangun model, ini menunjukkan bahwa lingkungan ini mungkin juga digunakan untuk tugas terkait `mediapipe`). Kemudian, library yang relevan seperti `os`, `tensorflow`, `keras`, `tensorflow_datasets`, `numpy`, `matplotlib`, dan `PIL` diimpor.
- 2. Hyperparameters:** Anda mendefinisikan beberapa hyperparameters penting seperti `learning_rate`, `label_smoothing_factor`, `batch_size`, `num_epochs`, `num_classes`, `patch_size`, `img_size`, dan `expansion_factor`. Hyperparameters ini mengontrol proses pelatihan dan arsitektur model.
- 3. MobileViT Utilities:**
 - o Anda mendefinisikan fungsi pembantu (`conv_block`, `correct_pad`, `inverted_residual_block`, `mlp`, `transformer_block`, `mobilevit_block`) yang merupakan blok bangunan dari arsitektur MobileViT.
 - o `conv_block`: Blok konvolusi dasar dengan aktivasi Swish.
 - o `correct_pad`: Fungsi utilitas untuk menghitung padding yang tepat, penting untuk downsampling.
 - o `inverted_residual_block`: Implementasi dari MobileNetV2 inverted residual block, digunakan untuk downsampling dan ekstraksi fitur lokal.
 - o `mlp`: Blok Multi-Layer Perceptron, digunakan dalam blok Transformer.
 - o `transformer_block`: Implementasi blok Transformer, digunakan untuk menangkap hubungan global antar patch.
 - o `mobilevit_block`: Ini adalah blok inti MobileViT yang menggabungkan konvolusi lokal (melalui `conv_block` dan `inverted_residual_block`) dan pemrosesan global (melalui `transformer_block`), kemudian menggabungkan (fuse) keduanya.
- 4. Model Architecture (`create_mobilevit`):**
 - o Fungsi `create_mobilevit` merangkai blok-blok utilitas untuk membangun model MobileViT (XXS variant) sesuai dengan skema yang ditunjukkan dalam gambar.
 - o Model dimulai dengan layer input dan normalisasi awal.
 - o Kemudian melewati serangkaian `conv_block`, `inverted_residual_block` (untuk downsampling dan fitur lokal), dan `mobilevit_block` (untuk menggabungkan fitur lokal dan global).
 - o Jumlah blok Transformer dan dimensi proyeksi di dalam setiap `mobilevit_block` ditentukan sesuai dengan varian XXS.
 - o Diakhiri dengan `GlobalAvgPool2D` dan layer Dense dengan aktivasi softmax untuk klasifikasi.
 - o Ringkasan model ditampilkan, menunjukkan jumlah parameter (sekitar 1.3 juta, yang sesuai untuk varian XXS).
- 5. Load & Prepare Dataset:**
 - o Kode ini (meskipun terganggu oleh `KeyboardInterrupt` di salah satu eksekusi) bertujuan untuk memuat data gambar dari Google Drive, membaginya menjadi set train dan validation, dan menyalinnya ke direktori terpisah.
 - o Anda menggunakan `ImageDataGenerator` dari `tensorflow.keras.preprocessing`

`img.image` untuk membuat generator data untuk training dan validation. Generator ini akan memuat gambar dalam batch dan melakukan penskalaan (`rescale=1./255`). Augmentasi data (rotasi, shift, shear, zoom, flip) dikomentari, artinya saat ini tidak digunakan.

- o `flow_from_directory` digunakan untuk membaca gambar langsung dari struktur folder (subfolder merepresentasikan kelas).

6. Model Training:

- o Model `mobilevit_xxs` yang dibuat sebelumnya dikompilasi dengan optimizer Adam, loss function `categorical_crossentropy`, dan metrik `accuracy`. Anda menggunakan `label_smoothing_factor` dalam loss function, yang dapat membantu mencegah overconfidence model.
- o Fungsi `run_experiment` mendefinisikan proses pelatihan. Ini membuat instance baru dari model, mengompilasinya, dan menggunakan `ModelCheckpoint` callback untuk menyimpan bobot model terbaik berdasarkan `val_accuracy`.
- o Model dilatih menggunakan metode `fit` dengan generator data training dan validation.

7. Evaluasi:

- o Kode ini mengevaluasi performa model yang telah dilatih.
- o Menggunakan `mobilevit_xxs.predict(val_gen)` untuk mendapatkan prediksi pada set validasi.
- o Menghitung `confusion_matrix` menggunakan label sebenarnya dari `val_gen.classes` dan prediksi model (`np.argmax(predictions, axis=1)`).
- o Memvisualisasikan confusion matrix menggunakan `ConfusionMatrixDisplay` dari scikit-learn, yang membantu melihat di kelas mana model berkinerja baik atau buruk.
- o Memplot grafik akurasi dan loss (training dan validation) selama epochs menggunakan data dari objek `history` yang dikembalikan oleh metode `fit`. Ini sangat berguna untuk memantau proses pelatihan dan mendeteksi overfit atau underfit.

8. Save Model:

- o Model disimpan dalam format Keras native (`.keras`) menggunakan `mobilevit_xxs.save()`.
- o Model juga dikonversi dan disimpan dalam format TensorFlow Lite (`.tflite`) untuk inferensi pada perangkat edge, menggunakan post-training dynamic-range quantization.

Analisis Keseluruhan:

- Kode ini menunjukkan pemahaman yang baik tentang cara membangun dan melatih model klasifikasi gambar menggunakan arsitektur modern seperti MobileViT di Keras.
- Penggunaan generator data (`ImageDataGenerator`) adalah praktik yang baik untuk menangani dataset gambar besar yang mungkin tidak muat sepenuhnya di memori.
- Implementasi blok-blok MobileViT terlihat benar sesuai dengan deskripsi arsitektur.
- Penggunaan `ModelCheckpoint` adalah krusial untuk menyimpan model terbaik selama pelatihan, terutama saat pelatihan berjalan lama dan performa validasi bisa berfluktuasi.
- Evaluasi dengan confusion matrix dan plot training/validation metrics memberikan wawasan yang baik tentang performa model